RAPPORT D'AUDIT DE PERFORMANCE

Simulateur de Maintien de l'Ordre

Optimisation d'un moteur de simulation temps réel écrit en Go

Système	**Simulation de manifestation / encerclement CRS**
Langage	**Go 1.26.4** (compilateur gc)
Auteur	**Marcus Declaude** — Master 2 Expert en Développement Logiciel (Dev B)
Date	**Octobre 2026**
Dépôt	github.com/Saurmc/opti-backend

Contexte de l'audit

Le système audité est un moteur de simulation temps réel : une foule d'environ 6 000 manifestants se déplace dans une trame de rues pour atteindre un objectif, pendant que des CRS tentent de l'encercler et d'interpeller. À chaque tick (cible 30 fois par seconde, soit un budget de **33,3 ms**), le moteur recalcule les interactions entre agents, met à jour les positions, diffuse l'état aux navigateurs connectés par WebSocket, puis enregistre les statistiques de la partie.

La première version fonctionne mais s'effondre sous les ~5 images par seconde : inutilisable en conditions réelles. L'objectif de cet audit est de ramener le moteur dans son budget, de mesurer chaque gain de façon reproductible, et de documenter les choix — y compris ceux qui n'ont pas marché. Le rapport est organisé en deux volets : d'abord les mesures, ensuite l'analyse des choix d'ingénierie et la tentative écartée.

Partie I — Mesures

Banc d'essai, protocole et résultats comparatifs

1.1 Banc d'essai & conditions de mesure

Composant	Spécification	Pourquoi ça compte
Processeur	Intel Core i7-13620H — 10 cœurs physiques (6 P + 4 E), 16 threads	Plafond de parallélisme : le pool de workers est calé sur `GOMAXPROCS` = 16
Caches	L1 ~80 Ko/cœur · L2 totale 9,7 Mo · L3 24 Mo	Les tableaux contigus (`[]Protester` , `scratch[]`) tiennent dans le cache et exploitent les lignes de 64 octets
Mémoire	16 Go	Large pour l'état manipulé ; le moteur est limité par le CPU, pas par la RAM
OS & runtime	Windows 11 (build 26200) · Go 1.26.4	Compilateur officiel, inlining et escape analysis actifs
Outillage	benchstat · Hyperfine · pprof · go test -race · Vegeta	Mesure statistique, débit binaire, profilage, détection de courses, charge HTTP

> **Conditions de test.** Toutes les mesures ont été prises sur la même machine **branchée sur secteur** (pas de bridage batterie), **aucun autre programme lancé** en parallèle, et **après un redémarrage** pour repartir d'un cache froid et d'un système au repos. Chaque benchmark CPU est précédé d'un préchauffage puis répété 10 fois ; les écarts ne sont retenus que si `benchstat` les juge significatifs ($p < 0{,}05$). Le moteur tourne en mode headless (`-bench` , sans affichage ni réseau) pour mesurer son débit réel, non plafonné par la boucle temps réel.

1.2 Ce qu'il faut tenir à chaque tick

Grandeur	Échelle	Coût dominant
Manifestants	6 000	Interactions de foule : en naïf, chaque agent regarde tous les autres → **36 M calculs/tick**
CRS	160	Perception et interpellation : ≈ 1 M comparaisons/tick
Carte	5000×5000	Navigation par flow-field (Dijkstra), recalculée périodiquement
Budget	33,3 ms	Cible 30 TPS ; la version initiale est 6× au-dessus
Frame réseau	~431 Ko	Diffusée à chaque tick : devient le goulot une fois le CPU réglé

1.3 Tableau comparatif : version initiale → version finale

Version	Ce qui change	ms/tick	Frame	Débit	Nature
V0	Version initiale : boucles imbriquées naïves, sérialisation JSON	204	431 Ko	5,6/s	baseline
V1	Grille spatiale : chaque agent ne regarde que ses 9 cases voisines	59	431 Ko	—	macro CPU

V2	Distances comparées au carré (on évite la racine)	36	431 Ko	28/s	micro CPU
écarté	Une goroutine par manifestant (voir Partie II)	**+17,6 %**	—	—	rejeté
V3	Pool de workers borné sur les cœurs + scan parallèle / décisions séquentielles	**8,8**	431 Ko	**114/s**	macro concurrence
V4	Frame WebSocket en binaire (5 octets/agent) au lieu de JSON	8,8	**41,8 Ko**	114/s	macro réseau
V5	Persistance SQLite : insertion en lot + index B-Tree	8,8	41,8 Ko	114/s	macro SQL

Entre V0 et V2, le gain CPU vaut **−82,5 %** (benchstat, p = 0,000, n = 10) ; l'ajout du pool (V3) retire encore **−71 %** (p = 0,000). Recoupé par Hyperfine sur le binaire : 10,29 s → 1,48 s sur 40 ticks, soit **×6,9**.

1.4 Les gains en un coup d'œil

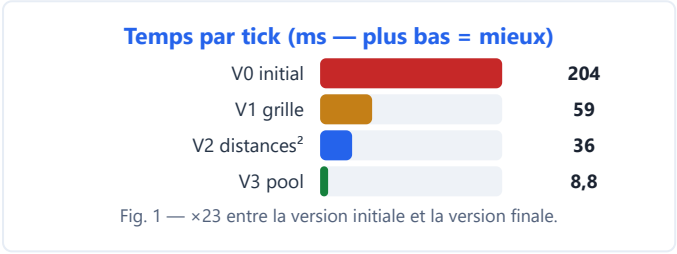


Fig. 1 — ×23 entre la version initiale et la version finale.



Fig. 2 — −90 % de bande passante par image.

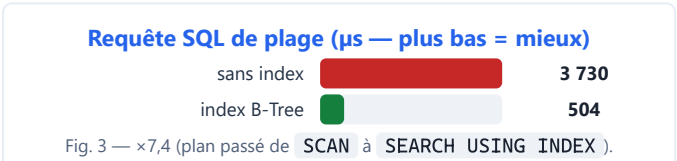


Fig. 3 — ×7,4 (plan passé de `SCAN` à `SEARCH USING INDEX`).



Fig. 4 — débit mesuré en headless (×20).

1.5 Comportement sous charge (Vegeta)

Le chemin de sérialisation a été exposé en HTTP (`/api/frame`) et bombardé à 2 000 requêtes/s pendant 15 s, pour vérifier la tenue en charge et regarder les percentiles plutôt que la moyenne.

Charge	Dispo.	Débit	P50	P90	P99	Octets/rép.
Vegeta — 2 000 req/s · 15 s	**100 %**	2 000,6/s	563 µs	1,68 ms	**24,5 ms**	41 308

La moyenne est bonne (P50 à 563 µs) mais le P99 grimpe à 24,5 ms : la queue vient du verrou global pris pendant la sérialisation (chaque requête sérialise 40 Ko sous le même mutex, qui sature vers 2 000 req/s). C'est une limite connue, à lever plus tard avec un snapshot sans verrou. Tout est rejouable en une commande : `make benchmarks` relance l'ensemble (env, benchstat, pprof, Hyperfine, Vegeta, SQL) et horodate les résultats.

Partie II — Analyse

Hot path, choix d'ingénierie et tentative écartée

2.1 Où partait le temps

Le profilage de la version initiale ne laisse pas de place au doute : presque tout le temps CPU est concentré dans la boucle de mise à jour, et plus précisément dans les calculs de distance entre agents. Ce n'est pas un problème de mémoire (34 Ko et ~1 000 allocations par tick, c'est négligeable pour le ramasse-miettes) : le moteur est limité par le calcul, pas par la RAM.



Fig. 5 — 99,5 % du temps dans une seule fonction : le voisinage en $O(n^2)$.

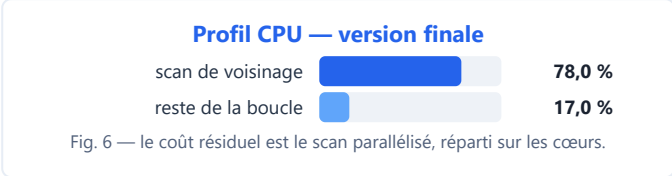


Fig. 6 — le coût résiduel est le scan parallélisé, réparti sur les cœurs.

La loi d'échelle confirme le diagnostic : passer de 4 000 à 6 000 agents (×1,5) multiplie le temps par 2,33, soit à peu près $1{,}5^2$ — la signature d'un coût en $O(n^2)$.

2.2 Les leviers actionnés

Calcul macro micro

On a remplacé le « chacun regarde tous les autres » par une **grille spatiale** : les agents sont rangés par cases, chacun ne compare qu'aux 9 cases autour de lui. Comme les interactions sont locales, le résultat est le même pour un coût qui passe de $O(n^2)$ à ~$O(n)$. En complément, on compare les distances **au carré** et on ne calcule la racine que lorsqu'on en a vraiment besoin.
Gain : −82,5 %.

Concurrence macro

La boucle a été **répartie sur les cœurs** avec un pool borné, en séparant le travail en deux temps : d'abord les scans de voisinage en parallèle (lecture seule, positions figées, donc sans conflit), puis les décisions et déplacements en séquentiel (ce qui garde le déterminisme). Vérifié avec le détecteur de courses de Go (aucune détectée).
Gain : −71 % (≈ ×3,5, limité par la partie séquentielle).

Réseau macro

La frame JSON (verbeuse, re-parsée côté client) a été remplacée par un **format binaire** : un agent tient en 5 octets, écrits directement dans un buffer réutilisé. Le petit reste (stats, décor) part en JSON dans un en-tête. Côté navigateur, lecture directe sans parsing de texte.
Gain : −90 % de bande passante.

Persistance macro

Les stats par tick sont écrites **en lot** (une transaction par partie, au lieu d'une écriture par ligne : ×1425). Pour la relecture, un **index B-Tree** sur `(run_id, tick_index)` fait passer une requête de plage d'un balayage complet à une recherche indexée, confirmé par `EXPLAIN QUERY PLAN`.
Gain : ×7,4.

2.3 Une optimisation qui a échoué

L'idée semblait logique : si paralléliser sur dix cœurs accélère, alors pousser le curseur au bout — **une goroutine par manifestant**, soit ~6 000 par tick — devrait aller encore plus vite, en laissant le runtime Go répartir tout ça.

En pratique c'est l'inverse : **+17,6 % de temps**. Créer 6 000 goroutines à chaque tick coûte plus cher que le travail lui-même (allocation de piles, ordonnancement), sans gagner le moindre parallélisme en plus — il n'y a que 10 cœurs. Et les 6 000 goroutines entremêlées cassent la lecture séquentielle des tableaux que le pool, lui, parcourt proprement.

Tentative	Idée	Résultat	Décision
Une goroutine par manifestant	Parallélisme « maximal »	+17,6 % (p=0,000)	écarté
Pool borné sur les cœurs	Une goroutine par cœur	référence	retenu

Mesuré à charge égale (benchstat, n = 10). On est revenu au pool ; le mode fautif n'est gardé que comme variante de benchmark pour appuyer ce constat.

Conclusion

Au départ, le moteur tournait à ~204 ms par tick, soit six fois la cible de 33,3 ms. À l'arrivée, il tient à **~8,8 ms** : **×23** plus rapide, un débit passé de 5,6 à 114 ticks/s, et une frame réseau divisée par dix. Le temps réel est désormais tenu avec de la marge.

L'accent a été mis sur le CPU, parce que c'est là qu'était le problème : presque tout le temps se jouait dans une poignée de lignes de calcul. La mémoire, elle, n'a jamais été limitante (quelques dizaines de Ko par tick) ; on l'a seulement effleurée avec des buffers réutilisés. Le travail terminé côté processeur, la pression est retombée sur le réseau, d'où le passage en binaire ; la base SQL a complété l'audit sur le volet persistance.

Le meilleur retour sur investissement vient de la grille spatiale. Ce simple changement d'algorithme (macro) efface à lui seul l'essentiel du coût, loin devant n'importe quel réglage fin. La règle est connue : on traite d'abord la complexité, on parallélise seulement après — jamais l'inverse.

La **tentative écartée** dit la même chose en négatif : à trop vouloir paralléliser, on a perdu 17,6 %. Le bon dosage n'est pas le plus agressif, mais celui qui épouse le nombre de cœurs réels de la machine.

Annexe — Gouvernance IA (`constitution.md`)

Un fichier `constitution.md` à la racine encadre les assistants de code IA, en quatre règles :

Règle	En clair
Posture	L'IA raisonne en ingénieur perf (budget tick, cache, allocations, cœurs) ; elle mesure au lieu de deviner.
Interdits	Sur le hot path : pas de `fmt.Sprintf`, pas de goroutines non bornées, pas de conversions `string↔[]byte` inutiles, pas d'alloc tas injustifiée, pas de racine quand le carré suffit.
Preuve	Chaque optimisation vient avec son hypothèse matérielle *et* la commande qui la vérifie. Sans preuve, c'est refusé.
Forme	Consignes courtes et vérifiables ; un commit par optimisation avec l'avant/après chiffré.